

Práctica N° 10: Transformaciones en el Espacio 3D Traslación, Rotación y Escalamiento

1. Objetivos

- Comprender e implementar las transformaciones geométricas tridimensionales básicas (traslación, rotación y escalamiento) en coordenadas homogéneas utilizando matrices de 4×4 .
- Aprender a transferir matrices de transformación desde la aplicación en C++ hacia el Vertex Shader mediante variables de tipo `uniform`.
- Utilizar la biblioteca GLM (OpenGL Mathematics) para la gestión eficiente de operaciones matemáticas y matriciales conforme a los estándares de OpenGL Moderno (Core Profile 3.3+).

2. Conceptos Teóricos

2.1. Coordenadas Homogéneas en 3D

Para representar transformaciones lineales y afines (como la traslación) mediante multiplicaciones matriciales uniformes, se añade una cuarta componente w a las coordenadas cartesianas tridimensionales $[x, y, z]^T$, obteniendo el vector en coordenadas homogéneas:

$$P = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

2.2. Traslación 3D

La traslación desplaza un punto de una posición $P_1 = [x_1, y_1, z_1]^T$ a una nueva posición $P_2 = [x_2, y_2, z_2]^T$ mediante un vector de desplazamiento $t = [t_x, t_y, t_z]^T$. Su expresión matricial en coordenadas homogéneas es:

$$\begin{bmatrix} x_2 \\ y_2 \\ z_2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ z_1 \\ 1 \end{bmatrix}$$

2.3. Escalamiento 3D con respecto al Origen

El escalamiento altera el tamaño de un objeto multiplicando sus coordenadas por factores de escala s_x, s_y, s_z . La matriz representativa es:

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.4. Rotación 3D sobre Ejes Principales

A diferencia del espacio bidimensional, las rotaciones en 3D se realizan alrededor de un eje. Las matrices de rotación para un ángulo θ sobre los ejes coordenados X, Y y Z son:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.5. Composición de Transformaciones y Rotación General

Para aplicar múltiples transformaciones, las matrices se multiplican en orden inverso al de aplicación geométrica (debido al modelado de vectores columna). Por ejemplo, para rotar un objeto alrededor de un eje arbitrario que no pasa por el origen, se debe:

1. Trasladar el objeto para que el eje de rotación pase por el origen (T^{-1}).
2. Alinear el eje arbitrario con uno de los ejes de coordenadas principales mediante rotaciones (R).
3. Efectuar la rotación deseada sobre dicho eje principal ($R_{eje}(\theta)$).
4. Aplicar las rotaciones inversas para restaurar la orientación original del eje (R^{-1}).
5. Aplicar la traslación inversa para devolver el eje a su posición inicial (T).

La matriz de transformación global resultante es:

$$M = T \cdot R^{-1} \cdot R_{eje}(\theta) \cdot R \cdot T^{-1}$$

3. Desarrollo de la Práctica

A continuación se presenta una implementación en OpenGL Moderno utilizando C++, GLFW para la gestión de ventanas, GLEW para la carga de extensiones y GLM para las operaciones matemáticas. Las transformaciones se calculan en la CPU y se inyectan en el Vertex Shader mediante variables `uniform`.

3.1. Código del Vertex Shader (shader.vert)

```
1 #version 333 core
2 layout (location = 0) in vec3 aPos;
3 layout (location = 1) in vec3 aColor;
4
5 out vec3 vertexColor;
6
7 uniform mat4 model;
8 uniform mat4 view;
9 uniform mat4 projection;
10
11 void main() {
```

```

12     gl_Position = projection * view * model * vec4(aPos, 1.0);
13     vertexColor = aColor;
14 }

```

Listing 1: Vertex Shader en GLSL 3.3

3.2. Código del Fragment Shader (shader.frag)

```

1 #version 333 core
2 in vec3 vertexColor;
3 out vec4 FragColor;
4
5 void main() {
6     FragColor = vec4(vertexColor, 1.0);
7 }

```

Listing 2: Fragment Shader en GLSL 3.3

3.3. Código de la Aplicación Principal (main.cpp)

```

1 #include <GL/glew.h>
2 #include <GLFW/glfw3.h>
3 #include <glm/glm.hpp>
4 #include <glm/gtc/type_ptr.hpp>
5 #include <iostream>
6 #include <vector>
7 #include <cmath>
8
9 const GLuint WIDTH = 800;
10 const GLuint HEIGHT = 600;
11
12 // CORREGIDO: Se cambi #version 333 por #version 330 core para
13 // compatibilidad en Linux
14 const char* vertexShaderSource = "#version 330 core\n"
15     "layout (location = 0) in vec3 aPos;\n"
16     "layout (location = 1) in vec3 aColor;\n"
17     "out vec3 vertexColor;\n"
18     "uniform mat4 model;\n"
19     "uniform mat4 view;\n"
20     "uniform mat4 projection;\n"
21     "void main() {\n"
22     "    gl_Position = projection * view * model * vec4(aPos, 1.0);\n"
23     "    vertexColor = aColor;\n"
24     "}\n";
25
26 const char* fragmentShaderSource = "#version 330 core\n"
27     "in vec3 vertexColor;\n"
28     "out vec4 FragColor;\n"
29     "void main() {\n"
30     "    FragColor = vec4(vertexColor, 1.0);\n"
31     "}\n";
32
33 //

```

```

33 // CONSTRUCCION MANUAL DE MATRICES (MODEL, VIEW, PROJECTION)
34 //
=====
35
36 glm::mat4 manualTranslate(float tx, float ty, float tz) {
37     return glm::mat4(
38         glm::vec4(1.0f, 0.0f, 0.0f, 0.0f),
39         glm::vec4(0.0f, 1.0f, 0.0f, 0.0f),
40         glm::vec4(0.0f, 0.0f, 1.0f, 0.0f),
41         glm::vec4(tx, ty, tz, 1.0f)
42     );
43 }
44
45 glm::mat4 manualScale(float sx, float sy, float sz) {
46     return glm::mat4(
47         glm::vec4(sx, 0.0f, 0.0f, 0.0f),
48         glm::vec4(0.0f, sy, 0.0f, 0.0f),
49         glm::vec4(0.0f, 0.0f, sz, 0.0f),
50         glm::vec4(0.0f, 0.0f, 0.0f, 1.0f)
51     );
52 }
53
54 glm::mat4 manualRotateX(float angle) {
55     float c = std::cos(angle);
56     float s = std::sin(angle);
57     return glm::mat4(
58         glm::vec4(1.0f, 0.0f, 0.0f, 0.0f),
59         glm::vec4(0.0f, c, s, 0.0f),
60         glm::vec4(0.0f, -s, c, 0.0f),
61         glm::vec4(0.0f, 0.0f, 0.0f, 1.0f)
62     );
63 }
64
65 glm::mat4 manualRotateY(float angle) {
66     float c = std::cos(angle);
67     float s = std::sin(angle);
68     return glm::mat4(
69         glm::vec4(c, 0.0f, -s, 0.0f),
70         glm::vec4(0.0f, 1.0f, 0.0f, 0.0f),
71         glm::vec4(s, 0.0f, c, 0.0f),
72         glm::vec4(0.0f, 0.0f, 0.0f, 1.0f)
73     );
74 }
75
76 glm::mat4 manualPerspective(float fovRadians, float aspect, float
nearPlane, float farPlane) {
77     float f = 1.0f / std::tan(fovRadians / 2.0f);
78     return glm::mat4(
79         glm::vec4(f / aspect, 0.0f, 0.0f, 0.0f),
80         glm::vec4(0.0f, f, 0.0f, 0.0f),
81         glm::vec4(0.0f, 0.0f, (farPlane + nearPlane) / (
nearPlane - farPlane), -1.0f),
82         glm::vec4(0.0f, 0.0f, (2.0f * farPlane * nearPlane) /
(nearPlane - farPlane), 0.0f)
83     );
84 }
85

```

```

86 //
87 // =====
88 // NUEVO: DIAGNOSTICO DE ERRORES DE COMPILACION DE SHADERS
89 // =====
89 void checkShaderErrors(GLuint shader, std::string type) {
90     GLint success;
91     GLchar infoLog[1024];
92     if (type != "PROGRAM") {
93         glGetShaderiv(shader, GL_COMPILE_STATUS, &success);
94         if (!success) {
95             glGetShaderInfoLog(shader, 1024, NULL, infoLog);
96             std::cerr << "ERROR::SHADER_COMPILATION_FAILED: " <<
type << "\n" << infoLog << std::endl;
97         }
98     } else {
99         glGetProgramiv(shader, GL_LINK_STATUS, &success);
100         if (!success) {
101             glGetProgramInfoLog(shader, 1024, NULL, infoLog);
102             std::cerr << "ERROR::PROGRAM_LINKING_FAILED: " << type
<< "\n" << infoLog << std::endl;
103         }
104     }
105 }
106
107 GLuint compileShaders() {
108     GLuint vertexShader = glCreateShader(GL_VERTEX_SHADER);
109     glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
110     glCompileShader(vertexShader);
111     checkShaderErrors(vertexShader, "VERTEX");
112
113     GLuint fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
114     glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
115     glCompileShader(fragmentShader);
116     checkShaderErrors(fragmentShader, "FRAGMENT");
117
118     GLuint shaderProgram = glCreateProgram();
119     glAttachShader(shaderProgram, vertexShader);
120     glAttachShader(shaderProgram, fragmentShader);
121     glLinkProgram(shaderProgram);
122     checkShaderErrors(shaderProgram, "PROGRAM");
123
124     glDeleteShader(vertexShader);
125     glDeleteShader(fragmentShader);
126     return shaderProgram;
127 }
128
129 int main() {
130     if (!glfwInit()) return -1;
131
132     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
133     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
134     glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
135     glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE); //
Requerido en entornos Linux/Mesa
136

```

```

137     GLFWwindow* window = glfwCreateWindow(WIDTH, HEIGHT, "Practica
10: Pipeline Manual Completo Linux", NULL, NULL);
138     if (!window) {
139         glfwTerminate();
140         return -1;
141     }
142     glfwMakeContextCurrent(window);
143
144     // Dimensionar expl citamente el Viewport para X11/Wayland
145     glViewport(0, 0, WIDTH, HEIGHT);
146
147     glewExperimental = GL_TRUE;
148     if (glewInit() != GLEW_OK) return -1;
149
150     glEnable(GL_DEPTH_TEST);
151     GLuint shaderProgram = compileShaders();
152
153     // Vertices de la Pir mide
154     float vertices[] = {
155         0.0f,  5.0f,  0.0f,   1.0f, 0.0f, 0.0f,
156        -5.0f, -5.0f,  5.0f,   0.0f, 1.0f, 0.0f,
157         5.0f, -5.0f,  5.0f,   0.0f, 0.0f, 1.0f,
158         5.0f, -5.0f, -5.0f,   1.0f, 1.0f, 0.0f,
159        -5.0f, -5.0f, -5.0f,   1.0f, 0.0f, 1.0f
160     };
161
162     unsigned int indices[] = {
163         0, 1, 2, 0, 2, 3, 0, 3, 4, 0, 4, 1, 1, 3, 2, 1, 4, 3
164     };
165
166     GLuint VAO, VBO, EBO;
167     glGenVertexArrays(1, &VAO);
168     glGenBuffers(1, &VBO);
169     glGenBuffers(1, &EBO);
170
171     glBindVertexArray(VAO);
172     glBindBuffer(GL_ARRAY_BUFFER, VBO);
173     glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);
174     glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
175     glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
GL_STATIC_DRAW);
176
177     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(
float), (void*)0);
178     glEnableVertexAttribArray(0);
179     glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(
float), (void*)(3 * sizeof(float)));
180     glEnableVertexAttribArray(1);
181
182     while (!glfwWindowShouldClose(window)) {
183         glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
184         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
185
186         glUseProgram(shaderProgram);
187
188         // A) Construcci n de la Matriz MODEL (M = T * R * S)
189         glm::mat4 T = manualTranslate(2.0f, 1.0f, 0.0f);

```

```

190     float timeValue = (float)glfwGetTime();
191     glm::mat4 Ry = manualRotateY(timeValue);
192     glm::mat4 Rx = manualRotateX(timeValue * 0.5f);
193     glm::mat4 R = Ry * Rx;
194     glm::mat4 S = manualScale(1.5f, 1.5f, 1.5f);
195
196     glm::mat4 model = T * R * S;
197
198     // B) Construcci n de la Matriz VIEW (V)
199     glm::mat4 view = manualTranslate(0.0f, 0.0f, -30.0f);
200
201     // C) Construcci n de la Matriz PROJECTION (P)
202     float fov = 45.0f * (3.14159265f / 180.0f);
203     float aspect = (float)WIDTH / (float)HEIGHT;
204     float nearPlane = 0.1f;
205     float farPlane = 100.0f;
206
207     glm::mat4 projection = manualPerspective(fov, aspect,
208     nearPlane, farPlane);
209
210     // Obtenci n e inyecci n de matrices uniform a la GPU
211     GLint modelLoc = glGetUniformLocation(shaderProgram, "model
212 ");
213     GLint viewLoc = glGetUniformLocation(shaderProgram, "view"
214 );
215     GLint projLoc = glGetUniformLocation(shaderProgram, "
216 projection");
217
218     glUniformMatrix4fv(modelLoc, 1, GL_FALSE, glm::value_ptr(
219 model));
220     glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(
221 view));
222     glUniformMatrix4fv(projLoc, 1, GL_FALSE, glm::value_ptr(
223 projection));
224
225     glBindVertexArray(VAO);
226     glDrawElements(GL_TRIANGLES, 18, GL_UNSIGNED_INT, 0);
227
228     glfwSwapBuffers(window);
229     glfwPollEvents();
230 }
231
232     glDeleteVertexArrays(1, &VAO);
233     glDeleteBuffers(1, &VBO);
234     glDeleteBuffers(1, &EBO);
235     glDeleteProgram(shaderProgram);
236
237     glfwTerminate();
238     return 0;
239 }

```

Listing 3: Programa Principal en C++ con OpenGL Moderno

4. Actividades / Ejercicio Propuesto

Modifique el programa proporcionado en la secci3n de desarrollo:

1. Un cubo gira a través de un ángulo de 30° alrededor de una línea que pasa por el origen $A = [0, 0, 0]$ y un punto $B = [7, 7, 7]$. Realice las operaciones de transformación para obtener el cubo girado.
2. Un tetrahedro debe ser rotado un ángulo de 45° alrededor de una línea que pasa a través del punto $[2,1,0]$ y $[6,5,0]$. Derive la matriz de transformación 3D requerida y muestre gráficamente dicha transformación.